EcoVerse | Viva preparation

Three-person guide: Frontend, Backend and Database Prepared 11 October 2026

Use these answers as speaking prompts, not a script to recite word for word. Each person should know their own layer and be able to explain one complete action across all three layers. The questions below are likely preparation topics, not a prediction of the exact viva.

Start together: introduce the project

"EcoVerse is a sustainability web application that motivates users through practical challenges. Users complete an environmental action and submit evidence. An administrator reviews it, and approved actions earn points, badges and progress in a virtual forest. We also provide community features and a directory of environmental organizations."

Project flow: Choose a challenge > Complete it > Submit proof > Admin review > Earn points > Update the forest.

Technical flow: Frontend > API request > Backend validation > Database > Response > Frontend update.

Person	Responsibility	Explain this example
1: Frontend	Pages, design, input, interactions and API calls	How a user finds and starts a challenge.
2: Backend	Authentication, validation, review and rewards	How a request becomes an authorized action.
3: Database	Tables, relationships, queries and transactions	How an attempt and its reward are stored.

Person 1 | Frontend

Opening: "I worked on the frontend, which is the part users see and interact with. We used HTML for structure, CSS for styling and responsiveness, and JavaScript for interactions and API communication. Vite handles development and production builds. Our main pages are My World, Discover, Challenges, Community and Connect. Three.js supports the virtual forest."

1. What is the frontend's responsibility?

It displays information, takes user input and communicates with the backend. For example, it shows challenge cards and sends a request when a user starts a challenge.

2. Why did you use HTML, CSS and JavaScript?

HTML provides structure, CSS controls presentation, and JavaScript adds behavior. They also let us demonstrate core web-programming concepts directly.

3. Did you use React?

No. Our current frontend uses HTML pages and JavaScript ES modules with Vite. Vite is a build tool, not a frontend framework.

4. What is Vite?

Vite provides a development server and creates the production frontend build. It processes JavaScript, CSS and assets for deployment.

5. What is the DOM?

The DOM is the browser's representation of an HTML page. JavaScript uses it to update the interface, such as inserting challenge cards after receiving data.

6. How do you get data from the backend?

We use the Fetch API. The frontend requests an API endpoint, receives JSON and updates the page. For example, fetch('/api/leaderboard') requests leaderboard data.

7. What are async and await?

They help handle operations that take time, such as API requests. Await pauses that asynchronous function until a result is available without blocking the whole browser.

8. How does challenge search work?

The challenge page uses search text and the selected category to filter the displayed challenge list. Categories include water, energy, waste, transport, lifestyle and nature.

9. How is the website responsive?

We use flexible layouts, CSS Grid and media queries. A multi-column card layout changes to fewer columns on smaller screens.

10. How do you handle errors?

We show loading, empty and error states. A failed request should display a useful message instead of pretending that an action succeeded.

11. What does Three.js do?

Three.js helps render the interactive 3D environment using WebGL. It manages scene objects such as trees and wildlife in the virtual forest.

12. What are GSAP and Lenis?

GSAP handles animations, and Lenis supports smooth scrolling. They improve presentation but do not control database updates or verified rewards.

13. What happens if 3D graphics are unavailable?

The interface has a fallback so users can still see their progress even if the browser cannot initialize the 3D scene.

14. Is hiding an admin button enough to secure the app?

No. Hiding buttons only changes the interface. The backend must verify the user's role before allowing an administrative action.

Person 2 | Backend

Opening: "I worked on the backend using Node.js and Express. It receives requests from the frontend, validates input, manages authentication and communicates with MySQL. It controls challenge attempts, evidence submission, administrator review and reward updates. Important rules stay on the server so users cannot award themselves points by changing browser code."

1. What is Node.js?

Node.js is a runtime that allows JavaScript to run outside the browser. We use it to run our server.

2. What is Express?

Express is a Node.js framework that helps define routes, process requests and send responses.

3. What is an API?

An API is an interface through which software components communicate. Our frontend uses API routes to request data or perform actions.

4. What is a route?

A route connects an HTTP method and URL to a handler. For example, GET /api/leaderboard runs the handler that returns leaderboard data.

5. Which HTTP methods do you use?

GET reads data, POST creates records or performs actions, PUT updates data, and DELETE removes data. Authentication and validation still determine whether an operation is permitted.

6. What is middleware?

Middleware runs during request processing. It can parse JSON, check a session, verify permissions or reject a request before the route handler runs.

7. What is authentication versus authorization?

Authentication checks who the user is. Authorization checks what that user may do. Signing in is authentication; checking permission to approve evidence is authorization.

8. How does login work?

The backend finds the user by email and checks the submitted password against the stored hash. If it matches, the server establishes an authenticated session and sends a session cookie.

9. Do you use JWT?

No. The current application uses server-side sessions with express-session and a MySQL session store.

10. What is stored in the cookie?

The cookie identifies the session. Actual session data is stored on the server in MySQL. We do not store the user's password in the cookie.

11. How are passwords protected?

We use salted scrypt hashing. During login, the backend derives a hash from the submitted password and compares it with the stored value.

12. Why hash rather than encrypt passwords?

Encryption can be reversed with a key. Password hashing supports checking a password without recovering the original password.

13. What happens when proof is submitted?

The backend validates the submission, processes the image, records its reference and changes the attempt to pending review. Submission alone does not award verified-action points.

14. Is verification AI-based?

No. An administrator reviews the evidence. The system provides checks and review signals, but it does not independently prove that an environmental action occurred.

15. What happens after approval?

The backend records approval and awards the associated points through a transaction. Updated progress is reflected in badges, the leaderboard and the forest.

16. How do you prevent duplicate rewards?

We check attempt status and eligibility on the server. The point ledger also has a unique completion reference, which helps prevent awarding the same attempt twice.

17. What is input validation?

It checks incoming data for the expected type, format and allowed values. We use Zod to validate request data.

18. What are rate limiting and Helmet?

Rate limiting restricts repeated requests to sensitive operations such as login. Helmet adds HTTP security headers. They support security but do not replace authentication or validation.

19. Why did 'service temporarily unavailable' appear?

In the incident we checked, Express was running but MySQL was stopped. The backend could not connect to database port 3307, so database-dependent requests failed.

20. Which status codes should you know?

200 means success; 201 means created; 400 means invalid request; 401 means authentication required; 403 means not permitted; 404 means not found; 409 means conflict; 429 means too many requests; 500 means server error.

Person 3 | Database

Opening: "I worked on the database layer using MySQL. It stores users, challenges, attempts, reviews, points, badges and community records. Related information is separated into tables and connected using primary and foreign keys. We use constraints and transactions to maintain consistency, especially when approving a challenge and awarding points."

1. Why did you choose MySQL?

Our data has clear relationships, such as users participating in challenges and receiving rewards. MySQL supports relational tables, constraints, joins and transactions.

2. What is a relational database?

It stores data in tables connected through keys. For example, a challenge attempt contains a user ID and a challenge ID.

3. What are the main tables?

Users stores accounts; challenges stores activity definitions; challenge_completions stores attempts; submission_reviews stores decisions; point_transactions stores rewards. Other tables support forest progress, badges, sessions, teams and community posts.

4. What is a primary key?

A primary key uniquely identifies each row. For example, users.id identifies one user.

5. What is a foreign key?

A foreign key references a record in another table. For example, challenge_completions.user_id references users.id.

6. Explain a relationship in your database.

One user can have many attempts, and one challenge can be attempted by many users. Users and challenges therefore have a many-to-many relationship through the attempts table.

7. Why separate challenges from completions?

A challenge describes a reusable task. A completion records one person's attempt. Keeping them separate avoids repeating the challenge definition for every user.

8. What is normalization?

Normalization organizes data to reduce unnecessary duplication and update problems. User details belong in users rather than being repeated in every attempt.

9. What is a JOIN?

A JOIN combines related information from tables. We join attempts with challenges to display a challenge title alongside the user's attempt status.

10. What is a transaction?

A transaction groups changes so they succeed or fail together. Approving a submission and recording its reward should not leave the system halfway updated.

11. What does rollback mean?

If a transaction fails, rollback cancels its uncommitted changes. This prevents an approval from being saved without the related reward update.

12. What is ACID?

Atomicity means all or none; consistency means constraints remain satisfied; isolation controls concurrent transactions; durability means committed changes persist.

13. How do you prevent SQL injection?

We use parameterized queries. User input is supplied as values rather than directly combined into SQL code.

14. What is an index?

An index helps the database find records efficiently. Our review queue has an index involving status and submission time.

15. How are evidence photos stored?

Processed image files are stored in a proof directory. MySQL stores the file reference and submission metadata, rather than the image itself.

16. Are organizations stored in MySQL?

Not currently. The Connect directory is curated data in server/organizations.js, served through an API. A database-backed directory management module is a possible improvement.

17. What happens if MySQL stops?

The frontend may still load, but features needing stored data fail. The backend cannot retrieve users, sessions or challenge records until the database is available.

18. What is a migration?

A migration creates or updates database structures in a controlled way. Our migration script prepares the schema and applies required changes.

Database quick reference

Table	What it stores
users / sessions	Accounts, password hashes, roles / login sessions
challenges / challenge_completions	Task definitions / individual attempts and their state
submission_reviews / point_transactions	Review history / traceable reward entries
forest_progress	Per-user progression and unlock values
badges / user_badges	Badge definitions / badges earned by a user
teams / team_members	Team definitions / membership
community_posts	Approved actions voluntarily shared with others

Example: joining attempts to challenge titles

SELECT ch.title, c.status FROM challenge_completions AS c JOIN challenges AS ch ON ch.id = c.challenge_id WHERE c.user_id = ?;

The question mark is a value placeholder supplied separately by the database driver. It is not a column name or text copied directly from user input.

Common questions | Everyone should know these

1. What makes EcoVerse useful?

It gives users a clear path from choosing an environmental action to recording and reviewing it. Progress feedback encourages repeat participation, and Connect helps users find relevant organizations.

2. Where is the game?

Game elements include points, badges, forest progression, wildlife unlocks and habitat management. They motivate participation around the real-world challenge workflow.

3. Does growing the forest mean actual trees are planted?

No. The forest is a virtual representation of progress. A digital tree should not be counted as a real planted tree.

4. Can users upload fake evidence?

Yes, that remains a limitation. Administrator review and duplicate-image signals improve accountability, but a photograph cannot guarantee that every claim is true.

5. What is the difference between Community and Connect?

Community supports participation inside EcoVerse, such as teams and shared actions. Connect directs users to external environmental organizations.

6. How are recommendations generated?

The current progress-based recommendations identify categories with little verified activity and no active attempts. They are rule-based, not machine-learning recommendations.

7. What is frontend versus backend validation?

Frontend validation gives quick feedback. Backend validation is essential because someone can bypass the interface and send a request directly.

8. What happens when I click Start Challenge?

The frontend sends an API request. The backend verifies the session and challenge eligibility, creates the attempt in MySQL and returns the result. The frontend displays the updated status.

9. Why can't the browser access MySQL directly?

That would expose database access and bypass application rules. The backend provides controlled access through authenticated, validated requests.

10. Is the project fully deployed?

The frontend is deployed on Vercel. The complete application works locally with MySQL. At the latest confirmed status, online backend, database and persistent upload storage are still pending.

11. How did you test it?

We tested backend workflows, progression and community behavior using automated tests. All 19 tests passed during local verification. We also checked the production build and captured the running interface.

12. What are the main limitations?

Evidence requires human review, online backend setup is unfinished, stock imagery needs refinement, and environmental estimates are not independently measured.

13. What would you improve next?

Complete backend hosting, add durable proof storage, improve moderation and maintain the organization directory. Document environmental estimate assumptions clearly.

14. Did you use AI tools?

Answer honestly: Yes, we used AI assistance during development and documentation. We reviewed and tested the application, and we can explain its workflow, database relationships and major modules. Do not claim personal contributions you did not make.

Group demonstration and last-minute revision

1. Frontend person: open the home page, explain navigation, search for a challenge and show its details.

2. Backend person: explain the start request, proof submission, pending review and administrator approval. Use an administrator account only if it has already been configured.

3. Database person: explain the attempt, review and reward records, then show how the dashboard reads updated progress. Avoid exposing passwords or private connection details.

Run locally in PowerShell

cd D:\Ecoversee

npm run db:local

npm start

Open http://localhost:3001 and keep the terminal open. These commands assume the existing configured project and initialized local database. If a database-dependent request fails, first check that MySQL is running. Do not reset or reseed your data immediately before the viva unless necessary.

Five things all three members must remember

Frontend: HTML/CSS/JavaScript with Vite, not React. Backend: Node.js and Express. Database: MySQL. Verification: administrator review, not AI. Authentication: server-side sessions, not JWT.

Practice one complete explanation: "I click Start Challenge. JavaScript sends a request. Express checks my session and the rules. MySQL stores my attempt. The server returns JSON. The page shows the new status."

Speak slowly, give one concrete example and stop after answering the question. If you do not know a detail, say what you do know and identify the module you would inspect rather than inventing an answer.

Project-specific facts follow the implementation and verification already reviewed for EcoVerse. Hosting status describes the latest confirmed state, not a new live deployment check.